LAMONT LABS · EXECUTION GOVERNANCE INFRASTRUCTURE

CerbaSeal-Core

v0.1.0 — System & Repository Breakdown

MAXIMUM DETAIL TECHNICAL REFERENCE

Generated: Fri, 05 Jun 2026 20:47:04 GMT

Jesse Lamont · jesse@lamontlabs.io

391/391	15/15	INV-01!’12	0 violations
TEST SUITE	**REPO AUDIT**	**INVARIANTS**	**IMPORT BOUNDS**
16 test files · zero regressions	all checks passing	100% test coverage	79 files scanned

CONTENTS

What CerbaSeal-Core Is

CerbaSeal-Core is a deterministic execution governance library for AI-integrated financial and enterprise workflows. It acts as a mandatory gate between any AI proposal and any consequential action. No payment, hold, escalation, or account modification may be released without passing through the gate. The gate is not advisory — it produces ALLOW, HOLD, or REJECT with a cryptographically chained audit trail that cannot be altered after the fact.

Core Design Principles

Fail-closed by default: Any unexpected error, malformed request, or missing precondition produces REJECT. There is no silent allow pa default-allow fallback.

AI is proposal-only: An AI actor can submit a proposal but cannot bear authority. Any attempt at self-authorization produces an uncondit REJECT via INV-05.

Human approval as a release precondition: For workflows requiring human approval, no ALLOW is issued until a valid, signed, correct present and verified.

Immutable decision envelopes: Every GateResult is registered in a module-private WeakSet. EvidenceBundleService rejects any result self-constructed results cannot bypass the gate.

Cryptographic hash chain: Every audit log entry hashes its own payload and links to the previous entry hash, forming a tamper-evident at any time.

Deterministic replay: Any past decision can be re-run through ReplayService to verify the same inputs produce the same output — prov logic drift over time.

Zero runtime dependencies: The enforcement core depends only on Node.js built-ins. No external npm packages are required at runtim

Architecture Layers

Layer	Source Path	Responsibility
Domain	src/domain/	Types, constants, errors, formatters, builders — structural definitions only, no business logic
Enforcement	src/services/execution/	ExecutionGateService — the mandatory evaluation gate. assertIsGateIssued() — WeakSet bypass guard
Audit	src/services/audit/	AppendOnlyLogService (in-memory) + FileBackedAppendOnlyLogService (JSONL). audit-hash-utils.ts — SHA-256 chain
Evidence	src/services/evidence/	EvidenceBundleService — assembles full decision context from gate result and audit events
Export	src/services/export/	ExportManifestService — point-in-time snapshot of evidence hashes for external transfer
Replay	src/services/replay/	ReplayService — re-runs past decisions through a fresh gate instance to verify determinism
Diagnostics	src/services/diagnostics/	DiagnosticReportService — operator-facing state analysis with recommended actions
Support	src/services/support/	OperatorActionService, SystemHealthService, SystemIntegrityService
Config	src/config/	cerbaseal-config.ts — loadCerbaSealConfig(), buildAllowedAuthorityClasses(). Reads cerbaseal.config.json
Examples	examples/	Working demos: browser-demo, consumer, agent-gate, auditor-view, support-readiness, 4 starter kits
Scripts	scripts/	repo-audit, export-proof, verify-proof, check-imports, check-invariants, generate-pilot-config, generate-evidence-report

Technology Stack

Component	Choice	Notes
Runtime	Node.js 18+	ESM module system ("type": "module" in package.json)
Language	TypeScript 5.6	Strict mode, target ES2022, module NodeNext
Test runner	Vitest 2.1	391 tests, 16 files, avg 7.7s full run
Script runner	tsx 4.21	Direct TypeScript execution without a compile step
PDF generation	pdfkit 0.18	devDependency — used only for report generation
Package manager	pnpm workspace	Monorepo with cerbaseal-demo React/Vite artifact
Demo UI	Vite + React	Browser review portal at /review, /pilot, /security, /deployment, /one-page

Primitive Union Types · src/domain/types/core.ts

Type Name	Members	Usage
AuthorityClass	system | ai | analyst | reviewer | manager | compliance_officer	Who is making the request. Validated at runtime. Extensible via config.
HumanAuthorityClass	analyst | reviewer | manager | compliance_officer	Subset used for ApprovalArtifact — excludes system and ai.
WorkflowClass	fraud_triage | transaction_escalation | account_hold_recommendation	Which workflow envelope governs the request.
ActionClass	allow | hold | reject | escalate | account_hold	Concrete action the gate may permit on ALLOW.
UnknownableActionClass	ActionClass | (string & {})	Input type on GovernedRequest. Gate validates and narrows or REJECTs.
ProposalSourceKind	ai | deterministic_rule	Origin of the proposal. Does not grant authority.
DecisionState	ALLOW | HOLD | REJECT	Final gate output. Produced once, immutable.

GovernedRequest · Primary gate input

Every field is validated before a decision is produced. Fields are checked in the order listed.

`requestId` `string` Non-empty. Unique identifier for this request. Used to derive all deterministic IDs.

`workflowClass` `WorkflowClass` Determines which workflows require unconditional approval (e.g. fraud_triage).

`jurisdiction` `string` Non-empty jurisdictional tag. Required for audit provenance.

`actorId` `string` Opaque identity of the actor submitting the request.

`actorAuthorityClass` `AuthorityClass` Validated at runtime against the allowedAuthorityClasses set — closes the gap for untyped callers.

`proposedActionClass` `UnknownableActionClass` Must exactly match proposal.requestedActionClass (INV-12).

`proposal` `DecisionProposal` The AI or rule-based proposal being evaluated (see below).

`sensitive` `boolean` If true, stale or invalid control status triggers REJECT (INV-08).

`prohibitedUse` `boolean` If true, unconditional REJECT regardless of other fields (INV-10).

`policyPackRef` `PolicyPackRef | null` Must be non-null. Carries id and version. (INV-01)

`provenanceRef` `ProvenanceRef | null` Must be non-null with modelVersion, ruleSetVersion, and sourceHash all non-empty. (INV-02)

`approvalRequired` `boolean` Caller-supplied flag. Some workflows override this to true unconditionally.

`approvalArtifact` `ApprovalArtifact | null` Must be present and valid when approval is required (INV-03).

`loggingReady` `boolean` Must be true. Prevents actions without an active audit trail (INV-04).

`controlStatus` `ControlStatus` Contains criticalControlsValid and stale. Checked for sensitive requests (INV-08).

`trustState` `TrustState` Contains trusted flag. Must be true (INV-09).

`createdAt` `string` ISO 8601 timestamp. Validated against approvalArtifact.approvedAt (INV-03).

DecisionEnvelope · Immutable gate output

Registered in the module-private WeakSet immediately after construction. Cannot be constructed outside evaluate().

`envelopeId` `string` Deterministic: 'env_' + requestId

`requestId` `string` Back-reference to the originating request

`workflowClass` `WorkflowClass` Echoed from the request

`finalState` `DecisionState` ALLOW | HOLD | REJECT — the gate's verdict

`permittedActionClass` `ActionClass | null` Set on ALLOW; null on HOLD or REJECT

`humanApprovalRequired` `boolean` Echoed from request.approvalRequired

`humanApprovalPresent` `boolean` True if approvalArtifact was present and valid

`proposalSourceKind` `ProposalSourceKind` Echoed from request.proposal.proposalSourceKind

`immutable` `true` Literal type — always true, cannot be false

`evidenceBundleId` `string` Deterministic: 'evidence_' + requestId

`trace.checkedInvariants` `InvariantCode[]` Every INV code checked during this evaluation

`trace.reasonCodes` `ReasonCode[]` Trigger code + terminal code (DECISION_ALLOWED / HELD / REJECTED)

`trace.evaluatedAt` `string` ISO timestamp of gate evaluation
`issuedAt` `string` ISO timestamp — when the envelope was issued

ApprovalArtifact · Human approval record

`approvalId` `string` Unique identifier for this approval record
`approverId` `string` Human reviewer identity (opaque string)
`forRequestId` `string` Must equal GovernedRequest.requestId — prevents cross-request reuse (INV-03 Fix 1)
`approverAuthorityClass` `HumanAuthorityClass` Must be analyst | reviewer | manager | compliance_officer
`privilegedAuthSatisfied` `boolean` Must be true — confirms the approver used privileged authentication
`immutableSignature` `string` Non-empty signature string — empty string triggers REJECT (INV-03)
`approvedAt` `string` ISO timestamp — must be >= request.createdAt (earlier timestamp = forged artifact)

EvidenceBundle · Full decision snapshot · src/domain/types/audit.ts

`evidenceBundleId` `string` Matches decisionEnvelope.evidenceBundleId
`request` `GovernedRequest` Deep-cloned original request — no shared reference
`decisionEnvelope` `DecisionEnvelope` Deep-cloned envelope — no shared reference
`releaseAuthorization` `ReleaseAuthorization | null` Present on ALLOW; null on HOLD/REJECT
`blockedActionRecord` `BlockedActionRecord | null` Present on HOLD/REJECT; null on ALLOW
`eventChain` `AuditLogEntry[]` All log entries for this requestId in insertion order
`createdAt` `string` ISO timestamp of bundle creation

AuditLogEntry · Hash-chained audit record

`eventId` `string` Unique entry identifier
`requestId` `string` Which request this event belongs to
`eventType` `AuditEventType` REQUEST_EVALUATED |

RELEASE_AUTHORIZED | ACTION_BLOCKED | EVIDENCE_BUNDLE_CREATED | EXPORT_MANIFEST_CREATED

`timestamp` `string` ISO timestamp of event recording

`payloadHash` `string` SHA-256 of the event payload JSON

`previousHash` `string | null` null for genesis entry; entryHash of the previous entry otherwise

`entryHash` `string` SHA-256 of (payloadHash + '|' + previousHash + '|' + timestamp)

SECTION 3

Invariant Registry — INV-01 through INV-12

Invariants are the non-negotiable rules of the system. Every check runs on every evaluate() call. A failed invariant throws a CerbaSealError that produces a controlled HOLD or REJECT — there is no path to ALLOW that skips any invariant. All 12 have explicit test coverage (verified by pnpm check:invariants, run as part of pnpm audit:repo).

INV-01 — NO_POLICY_PACK_NO_EXECUTION

!' REJECT

Every governed request must carry a reference to the policy pack under which it operates. Without a policy pack there is no basis for determining which rules govern the action — the gate has no mandate to permit anything.

Trigger:

request.policyPackRef === null

Reason code(s):

NO_POLICY_PACK

Test reference:

execution-gate-service.test.ts — 'rejects when no policy pack'

INV-02 — NO_PROVENANCE_NO_ACTION

!' REJECT

Full provenance (model version, rule set version, source hash) is required. Without it the decision cannot be audited or replayed deterministically. An empty string in any field is treated as missing.

Trigger:

provenanceRef === null OR any of modelVersion / ruleSetVersion / sourceHash is empty string

Reason code(s):

NO_PROVENANCE

Test reference:

execution-gate-service.test.ts — 'rejects when no provenance'

INV-03 — NO_REQUIRED_APPROVAL_NO_RELEASE

!' HOLD (missing) · REJECT (invalid/forged)

When approval is required — either by the caller flag OR by the workflow class (e.g. fraud_triage always requires approval regardless of the flag) — no ALLOW is issued unless a valid ApprovalArtifact is present. The artifact is validated on five dimensions: (1) forRequestId must equal requestId, (2) approvedAt must be >= request.createdAt, (3) approverAuthorityClass must be a valid HumanAuthorityClass, (4) privilegedAuthSatisfied must be true, (5) immutableSignature must be non-empty.

Trigger:

effectiveApprovalRequired && (artifact null OR artifact invalid)

Reason code(s):

REQUIRED_APPROVAL_MISSING / INVALID_APPROVAL_AUTHORITY / PRIVILEGED_AUTH_NOT_SATISFIED / APPROVAL_SIGNATURE_MISSING

Fixes applied:

Fix 1: request-binding check (forRequestId). Fix 2: WORKFLOWS_REQUIRING_APPROVAL set. Phase 6: timestamp anti-replay check.

Test reference:

execution-gate-service.test.ts — approval binding, timestamp, and forgery suites

INV-04 NO_LOGGING_NO_EXECUTION

!' REJECT

Before any action is released, the logging subsystem must be confirmed ready. Acting without an active audit trail is a governance failure — the gate will not permit it.

Trigger:

request.loggingReady === false

Reason code(s):

LOGGING_NOT_READY

Test reference:

execution-gate-service.test.ts — 'rejects when logging not ready'

INV-05 AI_NON_AUTHORITATIVE

!' REJECT

AI actors are proposal-only in two independent checks. Check A: if proposal.authorityBearing is true, REJECT — regardless of actor class. Check B: if the actor is 'ai' AND the proposal source is 'ai', REJECT — regardless of approvalRequired. Fix 4 closed a prior bypass where setting approvalRequired: false allowed an AI actor through check B.

Trigger:

proposal.authorityBearing === true OR (actorAuthorityClass === 'ai' AND proposalSourceKind === 'ai')

Reason code(s):

AI_CANNOT_AUTHORIZE

Fixes applied:

Fix 4: removed approvalRequired condition from AI authority check. The approval flag is irrelevant to whether AI has execution authority.

Test reference:

security/misuse-scenarios.test.ts, adversarial-integrity.test.ts

---

INV-06 NO_BYPASS_OF_EXECUTION_GATE

!' REJECT (thrown by assertIsGateIssued)

A module-private WeakSet (_gateIssuedResults) is populated only inside ExecutionGateService.evaluate(). EvidenceBundleService calls assertIsGateIssued() before accepting any result. A caller who constructs their own GateResult cannot add it to the WeakSet — the set is never exported and the reference is not accessible outside the module.

Trigger:

EvidenceBundleService.createBundle() called with a GateResult not in _gateIssuedResults WeakSet

Reason code(s):

MALFORMED_REQUEST

Fixes applied:

Fix 3: WeakSet introduced to close the path where a caller self-constructs a GateResult and feeds it directly to the evidence layer.

Test reference:

security/non-forgery.test.ts — 'rejects self-constructed gate result'

INV-07 IMMUTABLE_DECISION_ENVELOPE

!' Structural guarantee

DecisionEnvelope carries immutable: true as a TypeScript literal type. EvidenceBundleService deep-clones all objects before returning them. The envelope is never mutated after construction. This invariant is explicitly pushed to checkedInvariants[] on every path including ALLOW — it is always true.

Trigger:

Always — structural enforcement on every ALLOW/HOLD/REJECT path

Reason code(s):

N/A

Test reference: execution-gate-service.test.ts — immutable flag assertions across all paths

---

INV-08 **STALE_CONTROLS_BLOCK_SENSITIVE_RELEASE**
!' REJECT

For sensitive requests (e.g. high-value transactions, cross-border payments), the control status must be both current (not stale) and valid (critical controls passing). This prevents time-sensitive compliance controls — such as sanction list checks or credit limit verifications — from being bypassed by using an old control state.

Trigger:

request.sensitive === true AND (controlStatus.stale === true OR controlStatus.criticalControlsValid === false)

Reason code(s): CONTROL_STALE_OR_INVALID

Test reference: execution-gate-service.test.ts — stale controls tests

---

INV-09 **TRUST_STATE_REQUIRED**
!' REJECT

Every request must carry a valid trust state. If the system's trust evaluation has determined the request context is not trusted (e.g. flagged actor, revoked credentials, anomalous pattern), the gate rejects unconditionally — before approval state is even checked.

Trigger:

request.trustState.trusted === false

Reason code(s): TRUST_STATE_INVALID

Test reference: execution-gate-service.test.ts — 'rejects invalid trust state'

INV-10 **PROHIBITED_USE_MUST_BLOCK**
!' REJECT

If the request has been flagged as prohibited use — e.g. sanctioned jurisdiction, restricted action type, policy exclusion — the gate rejects unconditionally. This check runs after trust state but before approval state to ensure compliance blocks are absolute and cannot be overridden by providing an approval artifact.

Trigger:

request.prohibitedUse === true

Reason code(s): PROHIBITED_USE

Test reference: execution-gate-service.test.ts — 'rejects prohibited use'

INV-11 **REQUEST_SCHEMA_AND_ACTION_CLASS_VALID**
!' REJECT

Shape validation runs first, before any semantic checks. Catches: empty required string fields (requestId, jurisdiction, createdAt), empty proposal.reasonCodes[], unknown action classes (proposedActionClass or proposal.requestedActionClass not in ALLOWED_ACTION_CLASSES set), unknown authority classes (actorAuthorityClass not in allowedAuthorityClasses set). TypeScript enforces these at compile time; this check closes the runtime gap for JavaScript or JSON-deserialized callers.

Trigger:

requestId/jurisdiction/createdAt empty | proposal.reasonCodes empty | unknown action class | unknown authority class

Reason code(s): MALFORMED_REQUEST / UNKNOWN_ACTION_CLASS

Fixes applied: Fix 7: requestId non-empty validation added. Phase 5: runtime authority class validation added.

Test reference: execution-gate-service.test.ts — schema validation suite; security/contextual-boundary.test.ts

---

INV-12 **PROPOSAL_AND_REQUEST_ACTION_MUST_MATCH**
!' REJECT

The action class at the request level and the action class inside the proposal must be identical. This prevents a request from claiming one action type externally while the proposal contains a different action type — a potential vector for action-class confusion attacks where the outer request claims 'allow' but the proposal specifies 'escalate'.

Trigger:

request.proposedActionClass !== request.proposal.requestedActionClass

Reason code(s): INVALID_PROPOSAL

Test reference: execution-gate-service.test.ts — 'rejects mismatched action classes'

SECTION 4

Reason Code Registry — All 18 Codes

Reason codes appear in DecisionEnvelope.trace.reasonCodes[]. Every decision carries at least one code. HOLD and REJECT decisions always include a terminal code (DECISION_HELD or DECISION_REJECTED) in addition to the trigger code. All codes are defined in src/domain/constants/reason-codes.ts.

Reason Code	Final State	Invariant	Meaning
DECISION_ALLOWED	ALLOW	—	All checks passed. Action is released via ReleaseAuthorization.
DECISION_HELD	HOLD	—	Terminal suffix emitted on all HOLD outcomes.
DECISION_REJECTED	REJECT	—	Terminal suffix emitted on all REJECT outcomes.
NO_POLICY_PACK	REJECT	INV-01	policyPackRef is null — no policy basis for execution.
NO_PROVENANCE	REJECT	INV-02	provenanceRef is null or modelVersion / ruleSetVersion / sourceHash is empty.
REQUIRED_APPROVAL_MISSING	HOLD	INV-03	Approval is required (by flag or workflow class) but approvalArtifact is null.
INVALID_APPROVAL_AUTHORITY	REJECT	INV-03	Artifact bound to wrong requestId, invalid authority class, or approvedAt predates request.createdAt.
PRIVILEGED_AUTH_NOT_SATISFIED	REJECT	INV-03	privilegedAuthSatisfied is false on the approval artifact.
APPROVAL_SIGNATURE_MISSING	REJECT	INV-03	immutableSignature is an empty string on the approval artifact.
LOGGING_NOT_READY	REJECT	INV-04	loggingReady is false on the request.
AI_CANNOT_AUTHORIZE	REJECT	INV-05	AI actor attempted self-authorization, or proposal.authorityBearing is true.
PROHIBITED_USE	REJECT	INV-10	request.prohibitedUse is true.
CONTROL_STALE_OR_INVALID	REJECT	INV-08	Sensitive request with controlStatus.stale true or criticalControlsValid false.
TRUST_STATE_INVALID	REJECT	INV-09	request.trustState.trusted is false.
UNKNOWN_ACTION_CLASS	REJECT	INV-11	proposedActionClass or proposal.requestedActionClass is not a known ActionClass.
MALFORMED_REQUEST	REJECT	INV-11 / INV-06	Empty required field, self-constructed GateResult, or non-ISO timestamp.
INVALID_PROPOSAL	REJECT	INV-12	proposedActionClass does not identically match proposal.requestedActionClass.
AI_CANNOT_AUTHORIZE (auth-bearing)	REJECT	INV-05	proposal.authorityBearing is true — applies regardless of actor class.

SECTION 5

Execution Gate Service — Full Logic Flow

evaluate(request: GovernedRequest): GateResult

This is the single mandatory entry point for all governed actions. The method runs a strict ordered sequence of invariant assertions. The first failure short-circuits evaluation and produces a controlled HOLD or REJECT result. On success (all 14 steps pass), ALLOW is issued and registered in the module-private WeakSet.

Evaluation Sequence (strict order)

Step	Function	INV	On Fail	What Is Checked
1	assertRequestShape()	INV-11	REJECT	requestId, jurisdiction, createdAt non-empty; proposal.reasonCodes non-empty
2	assertActorAuthorityClass()	INV-11	REJECT	actorAuthorityClass in allowedAuthorityClasses set (runtime check — closes untyped caller gap)
3	assertKnownActionClass(proposedActionClass)	INV-11	REJECT	Must be one of: allow | hold | reject | escalate | account_hold
4	assertKnownActionClass(proposal.requestedActionClass)	INV-11	REJECT	Same check applied to the proposal-side action class
5	Action class match	INV-12	REJECT	proposedActionClass must identically equal proposal.requestedActionClass
6	assertPolicyPack()	INV-01	REJECT	policyPackRef must be non-null
7	assertProvenance()	INV-02	REJECT	provenanceRef non-null; all three fields non-empty strings
8	assertLoggingReady()	INV-04	REJECT	loggingReady must be true
9	assertProposalBoundary()	INV-05	REJECT	Not authority-bearing; AI actor + AI proposal = REJECT (unconditional)
10	assertProhibitedUse()	INV-10	REJECT	prohibitedUse must be false
11	assertControlStatus()	INV-08	REJECT	Sensitive: controls current and valid
12	assertTrustState()	INV-09	REJECT	trustState.trusted must be true
13	assertApprovalState()	INV-03	HOLD / REJECT	Effective approval required? Artifact present, bound, timestamped, signed, valid authority class?
14	buildDecisionEnvelope(ALLOW)	INV-07	ALLOW	Construct immutable envelope; add to WeakSet; build ReleaseAuthorization; return GateResult

Error Handling — Fail-Closed Guarantee

The try/catch at the top of evaluate() guarantees two things:

- CerbaSealError (expected failures): produces a clean DecisionEnvelope with the error's finalState (HOLD or REJECT), the trigger reason code, and the terminal suffix code (DECISION_HELD or DECISION_REJECTED).
- Non-CerbaSealError (unexpected exceptions): converted to a REJECT DecisionEnvelope with MALFORMED_REQUEST + DECISION_REJECTED codes. If even this fallback fails (the request is too broken to read), the original error is re-thrown — still fail-closed at the caller boundary, never silently allowing.

Deterministic IDs

All IDs are deterministic — replay produces the same identifiers for the same requestId

```
envelopeId          =  'env_'      + requestId
evidenceBundleId    =  'evidence_' + requestId
releaseAuthId       =  'release_'  + requestId
```

WORKFLOWS_REQUIRING_APPROVAL — Unconditional Override

Certain workflow classes require human approval unconditionally regardless of the caller-supplied approvalRequired flag.

The caller cannot opt out by setting approvalRequired: false.

```
// src/services/execution/execution-gate-service.ts
const WORKFLOWS_REQUIRING_APPROVAL = new Set<WorkflowClass>([
  'fraud_triage'   // approval always required; caller flag is treated as advisory only
]);

// In assertApprovalState():
const effectiveApprovalRequired =
  request.approvalRequired || WORKFLOWS_REQUIRING_APPROVAL.has(request.workflowClass);
```

assertIsGateIssued() — Bypass Prevention (INV-06)

```
// Module-private WeakSet — never exported, never accessible outside this file
const _gateIssuedResults = new WeakSet<object>();

// Registered only inside evaluate() on every outcome path (ALLOW, HOLD, REJECT)
_gateIssuedResults.add(result);

// Called by EvidenceBundleService before accepting any GateResult
export function assertIsGateIssued(result: GateResult): void {
  if (!_gateIssuedResults.has(result)) {
    throw new CerbaSealError({ ..., finalState: 'REJECT' });
  }
}
```

Core Authority Classes

Class	Kind	Can propose?	Can authorize?	Notes
system	System actor	Yes	Yes (non-AI path)	Internal system processes, deterministic rules
ai	AI actor	Yes — proposal only	NEVER	INV-05: AI actor + AI proposal = unconditional REJECT
analyst	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — valid approval authority
reviewer	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — valid approval authority
manager	Human approver	Yes	Yes (as approver)	HumanAuthorityClass — valid approval authority
compliance_officer	Human approver	Yes	Yes (as approver)	Highest human authority class

Runtime Extensibility — cerbaseal.config.json

New client-specific authority classes (e.g. 'risk_officer', 'supervisor', 'credit_committee') are added to cerbaseal.config.json. No TypeScript source code changes are required. The config is loaded at ExecutionGateService construction time via loadCerbaSealConfig().

```
// cerbaseal.config.json (current state — extended arrays are empty)
{
  "_comment": "Add client roles to extended[] — no TypeScript changes required.",
  "authorityClasses": {
    "core": ["system", "ai", "analyst", "reviewer", "manager", "compliance_officer"],
    "extended": []
  },
  "workflowClasses": {
    "core": ["fraud_triage", "transaction_escalation", "account_hold_recommendation"],
    "extended": []
  },
  "actionClasses": {
    "core": ["allow", "hold", "reject", "escalate", "account_hold"],
    "extended": []
  }
}
```

src/config/cerbaseal-config.ts — API Surface

```
interface GateConfig {
  additionalAuthorityClasses?: string[];
}

// Reads cerbaseal.config.json from process.cwd().
// Returns { additionalAuthorityClasses: config.authorityClasses.extended }.
// Falls back to {} if file not found (core classes only).
function loadCerbaSealConfig(): GateConfig

// Returns ReadonlySet<string> = CORE_AUTHORITY_CLASSES "* additionalAuthorityClasses
function buildAllowedAuthorityClasses(config?: GateConfig): ReadonlySet<string>
```

Usage Patterns

```
// Pattern 1: Default — core classes only (all existing tests unchanged)
const gate = new ExecutionGateService();

// Pattern 2: Inline extension
const gate = new ExecutionGateService({
  additionalAuthorityClasses: ['risk_officer', 'supervisor']
});

// Pattern 3: From cerbaseal.config.json
import { loadCerbaSealConfig } from './src/config/cerbaseal-config.js';
const gate = new ExecutionGateService(loadCerbaSealConfig());
```

IAuditLogService Interface

```
// src/services/audit/append-only-log-service.ts
interface IAuditLogService {
  append(event: AuditEventPayload): AuditLogEntry;   // write — no delete or update
  list(): AuditLogEntry[];                           // returns deep-cloned copy
  listByRequestId(requestId: string): AuditLogEntry[];
  verifyChain(): boolean;                            // re-computes all hashes
}
```

AppendOnlyLogService (in-memory)

Stores entries in a private array. Returns deep-cloned copies from all read methods — callers cannot mutate the internal log state. Suitable for development and short-lived processes.

FileBackedAppendOnlyLogService (persistent)

Same IAuditLogService interface. Writes every entry as a newline-delimited JSON record (JSONL) to a file path specified at construction using appendFileSync (synchronous — every write is durable). On construction, reads existing entries from the file to restore state and verifies chain integrity. Used by the fraud-workflow-starter and any production deployment requiring durable audit logs.

```
// Creates the file if it does not exist. Reads and restores state on construction.
const log = new FileBackedAppendOnlyLogService('./audit/fraud-triage.jsonl');
```

Hash Chain Construction · audit-hash-utils.ts

Three SHA-256 hashes per entry, computed using Node.js built-in crypto module:

```
payloadHash  =  SHA256( JSON.stringify(eventPayload) )
previousHash =  entryHash of the previous entry  (null string 'genesis' for first entry)
entryHash    =  SHA256( payloadHash + '|' + previousHash + '|' + timestamp )

// verifyChain() re-computes every entryHash from scratch and checks:
// (a) computed hash matches stored hash  (b) previousHash matches prior entry's entryHash
// Returns false on any mismatch — detects tampering, deletion, or reordering.
```

Events Written Per Request · EvidenceBundleService

Event Type	When Written	Key Payload Fields
REQUEST_EVALUATED	Always — on every evaluate() call	requestId, finalState, envelopeId
RELEASE_AUTHORIZED	On ALLOW only	requestId, releaseAuthorizationId, actionClass
ACTION_BLOCKED	On HOLD or REJECT	requestId, finalState, reasonCodes[]
EVIDENCE_BUNDLE_CREATED	Always — after the above events	requestId, evidenceBundleId

EvidenceBundleService.createBundle() — Full Flow

- 1. Call assertIsGateIssued(gateResult) — throws CerbaSealError if result not in WeakSet (INV-06)
- 2. Append REQUEST_EVALUATED event to audit log
- 3. If releaseAuthorization present: append RELEASE_AUTHORIZED event
- 4. If blockedActionRecord present: append ACTION_BLOCKED event

- 5. Append EVIDENCE_BUNDLE_CREATED event
- 6. Call logService.listByRequestId(requestId) — retrieve the full event chain for this request
- 7. Assemble EvidenceBundle with deep-clones of all objects (no shared references to internal state)
- 8. Return a deep-clone of the bundle — double-cloned to prevent any mutation of internal log state

ExportManifestService

Creates a point-in-time snapshot of evidence hashes for external transfer. The manifest captures sourceEventHashes[] (one per AuditLogEntry) but does not include the request payload — suitable for sharing with parties who need proof of decision without the underlying data.

```
interface ExportManifest {
  manifestId: string;
  evidenceBundleId: string;
  requestId: string;
  envelopeId: string;
  exportedAt: string;
  exportType: 'authority_package';
  sourceEventHashes: string[];       // one hash per AuditLogEntry
  sourceEvidenceImmutable: true;     // literal type — always true
}
```

ReplayService · src/services/replay/

Takes a past EvidenceBundle and re-runs the original GovernedRequest through a fresh ExecutionGateService instance. Compares replayed finalState against original finalState. Used to verify: (a) the gate is deterministic, (b) logic has not drifted since the original decision, (c) the evidence bundle faithfully represents what happened.

```
interface ReplayResult {
  originalRequestId: string;
  originalEnvelopeId: string;
  replayedFinalState: 'ALLOW' | 'HOLD' | 'REJECT';
  replayedPermittedActionClass: string | null;
  matchedOriginalOutcome: boolean;   // true iff replayedFinalState === original finalState
  replayedAt: string;
}
```

DiagnosticReportService · src/services/diagnostics/

Generates a structured operator-facing diagnostic report from an EvidenceBundle. Includes system state snapshot, invariant check summary, recommended actions (e.g. 'obtain human approval', 'check control status'), and a risk assessment. Used by the browser-demo support-readiness view.

```
interface DiagnosticReport {
  reportId: string;
  evidenceBundleId: string;
  systemState: SystemState;
  invariantSummary: InvariantCheckSummary[];
  recommendations: string[];
  riskAssessment: string;
  generatedAt: string;
}
```

SystemHealthService · src/services/support/

Checks system preconditions: logging ready, control status valid, trust state valid, audit chain integrity. Returns a HealthReport with individual check results and an overall status. Used by the /health route in the REST API starter kit.

SystemIntegrityService · src/services/support/

Runs integrity checks across the audit log chain. Verifies all entries are chain-linked, no gaps exist, and hash values are consistent. Used as a background health check and on export to confirm evidence is untampered before sharing.

OperatorActionService · src/services/support/

Provides operator-level actions: acknowledge a blocked decision, escalate a held request, request re-evaluation, generate a support ticket. All actions are recorded to the audit log.

CertificateFormatter · src/domain/formatters/

Generates a human-readable governance certificate from an EvidenceBundle. Used by the auditor-view example to render a printable compliance record showing the decision, approver identity, invariants checked, and audit chain summary.

SECTION 9

Test Suite — 16 Files, 391 Tests

All tests run with Vitest 2.1. Full suite completes in ~7.7 seconds. Zero mocks of the enforcement core — every test uses real ExecutionGateService instances.

Test File	~Tests	Coverage Focus
test/execution-gate-service.test.ts	~120	All 12 invariants; every ALLOW/HOLD/REJECT path; approval binding; timestamp validation; authority class validation; schema edge cases
test/adversarial-integrity.test.ts	~18	Advanced attack scenarios: approval replay across requests, cross-request binding, AI authority escalation, malformed envelope injection
test/audit-evidence-export.test.ts	~30	AppendOnlyLogService chain construction and verification; EvidenceBundleService assembly; ExportManifestService output structure
test/persistent-audit-log.test.ts	~20	FileBackedAppendOnlyLogService: write, read-back, chain verify across restarts, multi-request interleaving
test/diagnostic-report-service.test.ts	~18	DiagnosticReportService output structure; recommendation generation for each failure mode; risk assessment accuracy
test/snapshots/enforcement-loop.snapshot.test.ts	~8	Full ALLOW/HOLD/REJECT flows serialized to snapshots and compared across runs — detects output shape changes
test/security/fail-closed.test.ts	2	Unexpected non-CerbaSealError exceptions produce REJECT — the gate never silently allows on error
test/security/non-forgery.test.ts	2	Self-constructed GateResult is rejected by assertIsGateIssued() — WeakSet bypass prevention (INV-06)
test/security/misuse-scenarios.test.ts	~15	AI self-authorization; authority-bearing proposals; approval authority class forgery; privilege escalation attempts
test/security/contextual-boundary.test.ts	~12	Boundary cases: empty strings, null fields, zero-length arrays, whitespace-only values, ISO timestamp edge cases
test/integration/full-flow.test.ts	1	End-to-end: request !’ evaluate !’ evidence bundle !’ replay !’ verify chain — single comprehensive flow test
test/integration/system-integration.test.ts	1	Full stack integration: gate + audit + evidence + diagnostic + health check + export
test/integration/browser-demo-routes.test.ts	~40	All 9 demo HTTP server routes respond 200 with correct JSON structure and scenario outcomes
test/integration/review-portal-routes.test.ts	~40	Review portal routes: /review, /pilot, /security, /deployment, /one-page — content assertions
test/integration/support-readiness.test.ts	~30	Support readiness demo: all operator action scenarios, health check outcomes, diagnostic report structure
test/integration/external-signal-examples.test.ts	~34	External signal injection: trust state overrides, control status variations, prohibited use, provenance permutations

Test Commands

```
pnpm test              # run all 391 tests once (~7.7s)
pnpm test:watch        # watch mode — re-runs on file changes
pnpm check:invariants  # verify 12/12 invariants have covering tests
pnpm audit:repo        # run all 15 repo audit checks (includes test count verification)
```

SECTION 10

Repo Audit — 15 Checks Explained

pnpm audit:repo runs scripts/repo-audit.ts — 15 automated checks that verify the repo is in a releasable, self-consistent state. All 15 currently pass. The script exits non-zero on any failure.

#	Check Name	What It Verifies
1	Full test suite passes	Runs pnpm test — expects all 391 tests to pass with zero failures
2	TypeScript compiles without errors	Runs tsc --noEmit — expects zero type errors across all source files
3	README anchor strings present	Checks for 4 required anchor strings in README.md
4	All portal routes respond 200	Starts the browser-demo HTTP server, requests all 9 routes, expects HTTP 200 on each
5	No src/ file unreferenced in tests or examples	Every .ts file under src/ must appear in at least one test or example file
6	Invariant registry non-empty	Loads src/domain/constants/invariants.ts — verifies 12 invariants are defined
7	Known-limitations section in README	README.md must contain a '## Known Limitations' heading
8	Test count in README matches actual	The '391 tests' claim in README must match the actual Vitest run count
9	demo:web:validate passes	Runs examples/browser-demo/validate-demo.ts — all browser demo assertions pass
10	demo:support:validate passes	Runs examples/support-readiness/validate-support-readiness.ts — 13 assertions pass
11	review:validate passes	Runs examples/browser-demo/validate-review-portal.ts — 110 assertions pass
12	No import boundary violations	Runs scripts/check-imports.ts — 0 violations across 79 files scanned
13	All invariants linked to covering tests	Runs scripts/check-invariant-coverage.ts — 12/12 invariants have test coverage
14	No stale test-count references in docs	All documentation references to test counts match the actual current count
15	Pilot documents exist with required sections	3 pilot documents verified for required section headings

Import Boundary Rules · scripts/check-imports.ts

The following boundaries are enforced across all 79 scanned files:

- examples/ and scripts/ must not import from test/
- src/ must not import from examples/, scripts/, or test/
- test/ must not import from examples/ or scripts/ (except for shared fixtures)
- No circular dependencies within src/services/

Proof Snapshot · scripts/export-proof.ts

pnpm export:proof generates docs/reports/proof-snapshot.json — a signed manifest of the repo state including test suite results, audit check results, invariant count, git commit hash, branch, and a SHA-256 manifestChecksum of the manifest body. The snapshot can be independently verified with pnpm verify:proof. The generate:evidence-report script reads this snapshot to produce the compliance evidence package.

```
pnpm export:proof             # generate docs/reports/proof-snapshot.json
pnpm verify:proof             # verify the snapshot's manifestChecksum
pnpm generate:evidence-report  # generate 3-file compliance evidence package
```

SECTION 11

CLI Scripts & Commands

Script	Entry File	Purpose
pnpm test	vitest	Run all 391 tests
pnpm test:watch	vitest	Watch mode — re-runs on file changes
pnpm typecheck	tsc --noEmit	TypeScript type check without emitting JS
pnpm demo	examples/run-demo.ts	Console demo of all gate scenarios
pnpm demo:web	examples/browser-demo/server.ts	Start browser-based review portal (HTTP, reads $PORT)
pnpm demo:web:validate	examples/browser-demo/validate-demo.ts	Assert all demo routes and scenario outcomes
pnpm demo:consumer	examples/consumer-example/consumer.ts	Consumer pattern demo
pnpm demo:consumer:validate	examples/consumer-example/validate-consumer.ts	Assert consumer demo outcomes
pnpm demo:agent	examples/agent-gate/run-agent-gate.ts	AI agent gate demo
pnpm demo:agent:validate	examples/agent-gate/validate-agent-gate.ts	Assert agent gate demo outcomes
pnpm demo:audit	examples/auditor-view/run-auditor-view.ts	Auditor certificate view demo
pnpm demo:audit:validate	examples/auditor-view/validate-auditor-view.ts	Assert auditor view outcomes
pnpm demo:support	examples/support-readiness/run-support-readiness.ts	Support readiness demo
pnpm demo:support:validate	examples/support-readiness/validate-support-readiness.ts	Assert support readiness outcomes (13 assertions)
pnpm review:validate	examples/browser-demo/validate-review-portal.ts	Assert 110 review portal scenarios
pnpm demo:all	(all demos)	Run all demos sequentially
pnpm check:imports	scripts/check-imports.ts	Verify import boundary rules across 79 files
pnpm check:invariants	scripts/check-invariant-coverage.ts	Verify 12/12 invariants have test coverage
pnpm export:proof	scripts/export-proof.ts	Generate docs/reports/proof-snapshot.json with SHA-256 checksum
pnpm verify:proof	scripts/verify-proof.ts	Verify proof-snapshot.json manifestChecksum
pnpm audit:repo	scripts/repo-audit.ts	Run all 15 repo audit checks
pnpm generate:pilot-config	scripts/generate-pilot-config.ts	Generate pilot config package from wizard-input.json
pnpm generate:evidence-report	scripts/generate-evidence-report.ts	Generate evidence package from proof-snapshot.json
pnpm generate:system-pdf	scripts/generate-system-pdf.ts	Generate this PDF

generate:pilot-config — Output Files · Written to pilot-config/

cerbaseal-config.json:
Ready-to-use runtime config for the client's workflow and authority classes

pilot-checklist.md:
Phase-by-phase onboarding checklist with owner and acceptance criteria per item

scenario-test.ts:
Runnable TypeScript test of the client's specific workflow (approve and reject scenarios)

deployment-summary.md:
One-page deployment summary with environment variables, pre-flight checklist, and next steps

generate:evidence-report — Output Files · Written to evidence-package/

governance-summary.md:
Narrative compliance evidence: test results, audit checks, invariant coverage, chain validity

decision-summary.json:
Machine-readable enforcement state: gate config, invariants, proof snapshot metadata

audit-integrity-summary.md:
Hash chain verification details: manifest checksum status, HMAC signature status

SECTION 12

Integration Starter Kits (×4)

Four complete working examples in examples/. Each runs standalone with pnpm tsx. Each has a README.md explaining the pattern and how to adapt it to a specific client workflow.

rest-api-starter · examples/rest-api-starter/

Pattern: HTTP REST wrapper — expose the gate as an API service

A plain Node.js HTTP server (no framework) that wraps ExecutionGateService as REST endpoints. Use as a sidecar or internal service that other systems call over HTTP.

- POST /evaluate — accept a GovernedRequest JSON body, return GateResult + EvidenceBundle
- GET /decisions — list all decisions from the in-memory audit log
- GET /decisions/:id — retrieve a specific decision by requestId
- GET /health — system health check (logging ready, chain valid, control status)
- sample-request.json — ready-to-use example request body for testing

```
pnpm tsx examples/rest-api-starter/index.ts
```

financial-approval-starter · examples/financial-approval-starter/

Pattern: AI proposes !' analyst holds !' manager approves !' ALLOW

Demonstrates the two-step human approval pattern for financial escalations. Shows the full HOLD !' ALLOW lifecycle with EvidenceBundle generation and AI self-authorization prevention.

- Step 1: Submit without approval !' HOLD (REQUIRED_APPROVAL_MISSING)
- Step 2: Manager reviews and constructs ApprovalArtifact with privilegedAuthSatisfied: true
- Step 3: Resubmit with artifact !' ALLOW + ReleaseAuthorization
- Step 4: Generate EvidenceBundle with 3-event hash chain
- Step 5: Verify AI alone !' REJECT (AI_CANNOT_AUTHORIZE) — demonstrates unconditional enforcement

```
pnpm tsx examples/financial-approval-starter/index.ts
```

fraud-workflow-starter · examples/fraud-workflow-starter/

Pattern: AI-scored triage with persistent file-backed audit log

Processes transactions through risk scoring (0–100) with risk-level routing. Uses FileBackedAppendOnlyLogService for durable JSONL audit logs. Demonstrates fraud_triage unconditional approval enforcement.

- Low risk (< 50): allow action — note: fraud_triage workflow always requires approval (HOLD produced regardless)
- Medium risk (50–79): hold action, analyst approval required !' resubmit !' ALLOW
- High risk ("e 80): escalate action, compliance_officer approval required !' resubmit !' ALLOW
- Audit log: ./audit/fraud-triage.jsonl (JSONL format, persists across runs)
- 15 total audit entries generated across 3 transactions with verified hash chain

```
pnpm tsx examples/fraud-workflow-starter/index.ts
```

agent-integration-starter · examples/agent-integration-starter/

Pattern: AI agent proposes !' system actor carries !' human approves

Shows the correct pattern for integrating an AI agent: the AI never submits its own proposal as actor. Instead a system actor carries the AI proposal. Includes a deliberate wrong-pattern demo showing why AI self-authorization always produces REJECT.

- Scenario 1 (wrong pattern): AI actor submits own proposal !' REJECT (AI_CANNOT_AUTHORIZE, INV-05)
- Scenario 2 (correct pattern): System actor carries AI proposal !' HOLD (waiting for human review)
- Scenario 3: Human reviewer approves !' ALLOW + EvidenceBundle with 3-event chain
- Evidence includes: model version, reviewer identity, approver authority class
- Explicit summary of integration boundary rules printed on each run

```
pnpm tsx examples/agent-integration-starter/index.ts
```

SECTION 13

Adoption Layer — 5 Priorities

Five deliverables built in response to the Line Axia audit. Together they allow a client to onboard, configure, pilot, and generate compliance evidence without requiring Jesse to be present at every step.

1 Authority Class Registry

cerbaseal.config.json · src/config/cerbaseal-config.ts

New client authority classes added to cerbaseal.config.json extended array — zero TypeScript source code changes required. ExecutionGateService constructor accepts optional GateConfig. Default (no-args) constructor uses core classes only, making this a zero-breaking-change addition. buildAllowedAuthorityClasses() merges core + extended into a ReadonlySet used at runtime.

!’ Eliminates: 'Jesse must update the code every time we add a new role to the client's workflow'

2 Integration Starter Kits

examples/rest-api-starter/ · examples/financial-approval-starter/ · examples/fraud-workflow-starter/ · examples/agent-integration-starter/

Four complete working examples covering the most common integration patterns (REST API wrapper, financial approval flow, AI-scored fraud triage, AI agent governance). Each runs standalone with pnpm tsx, includes a README with adaptation guidance, and demonstrates the correct actor/approver boundary. Client developers can copy and adapt without needing to understand the full codebase.

!’ Eliminates: 'We need Jesse on a call to show us how to integrate CerbaSeal'

3 Workflow Config Generator

scripts/generate-pilot-config.ts · scripts/wizard-input.example.json

Client fills in wizard-input.json (a questionnaire covering workflow name, actions, AI system description, approver roles, approval rules, evidence config, and deployment environment). pnpm generate:pilot-config reads the file and produces a complete 4-file pilot configuration package: cerbaseal-config.json, pilot-checklist.md, scenario-test.ts, deployment-summary.md — all tailored to the client's workflow.

!’ Eliminates: 'Jesse must manually configure CerbaSeal for each client deployment'

4 Pilot Evidence Package Generator

scripts/generate-evidence-report.ts

After a successful pilot, run pnpm export:proof (generates cryptographically verified proof-snapshot.json) then pnpm generate:evidence-report to produce a 3-file compliance evidence package: governance-summary.md (narrative), decision-summary.json (structured data), and audit-integrity-summary.md (hash chain verification). The manifest checksum is verified before output is produced.

!’ Eliminates: 'Jesse must produce the compliance evidence package manually for each client'

5 Founder Independence Kit

docs/FOUNDER-INDEPENDENCE-KIT.md · docs/client-adoption/onboarding-sequence.md

FOUNDER-INDEPENDENCE-KIT.md is the master index covering all 7 adoption stages from client qualification through compliance review. onboarding-sequence.md is an 8-phase sequence (Phase 0 through Phase 8) that a client follows end-to-end without Jesse. Includes an escalation table (what to escalate vs. self-serve), integration starter kit selection guide, quick command reference, and an explicit list of what still requires Jesse.

!’ Eliminates: 'The entire sales and onboarding process depends on Jesse's availability'

Browser Demo Server · examples/browser-demo/server.ts

A plain Node.js HTTP server (no framework) serving both HTML portal pages and JSON API endpoints. Start with: pnpm demo:web (reads PORT environment variable, defaults to 3000). All 9 routes are tested in test/integration/browser-demo-routes.test.ts.

Portal Pages — HTML for human review

Route	Page Title	Content
/review	CerbaSeal Review Portal	Interactive governance decision review with live scenario evaluation
/pilot	Pilot Readiness	Pilot readiness checklist and deployment status dashboard
/security	Security Summary	Invariant list, adversarial test results, security posture overview
/deployment	Deployment Guide	Step-by-step deployment instructions and environment configuration
/one-page	One-Page Summary	Executive-level one-page overview of the CerbaSeal governance model

Scenario APIs — JSON, return real gate decisions

Route	Scenario	Expected Outcome
POST /api/reject	AI actor attempts self-authorization	REJECT — AI_CANNOT_AUTHORIZE (INV-05)
POST /api/hold	System actor, approval required but missing	HOLD — REQUIRED_APPROVAL_MISSING (INV-03)
POST /api/allow	System actor, valid approval artifact present	ALLOW — DECISION_ALLOWED (all invariants pass)

Data APIs — JSON, supply data to the portal pages

Route	Returns
/api/review-summary	Recent decisions, hash chain validity status, invariant check summary for the review portal
/api/pilot-readiness	Pilot checklist items with status, deployment mode, configuration summary for the pilot page
/api/security-summary	Invariant list with test coverage, adversarial test results, overall security score for the security page

Validation Scripts

```
pnpm demo:web:validate   # validate-demo.ts — exercises all routes, checks JSON structure and outcomes
pnpm review:validate     # validate-review-portal.ts — 110 assertions across all portal routes
```

cerbaseal-demo Artifact · Vite + React

A separate React/Vite application in artifacts/cerbaseal-demo/ provides a production-grade browser UI. Registered as a pnpm workspace artifact with its own dev server. Start with: pnpm --filter @workspace/cerbaseal-demo run dev

Package name: @workspace/cerbaseal-demo

Dev server: Vite — reads PORT environment variable to avoid port collisions

Key files: src/hooks/use-toast.ts, src/lib/utils.ts

Config: vite.config.ts, tsconfig.json, components.json

SECTION 15

Full Repository File Tree

All source, test, script, documentation, and configuration files. node_modules excluded.

/ (root)

`package.json` Root package — 24 scripts, tsx + pdfkit + vitest devDependencies, ESM module type

`tsconfig.json` Strict TypeScript — ES2022 target, NodeNext module, strict: true, exactOptionalPropertyTypes: true

`cerbaseal.config.json` Runtime authority/ workflow/action class registry with core and extended arrays

`vitest.config.ts` Vitest configuration — include: test/**/*.test.ts

`.gitignore` Excludes: wizard-input.json, pilot-config/, evidence-package/, audit/, dist/, .env

`README.md` Full public README with system overview, adoption layer section, 391 test count

`CERBASEAL_PILOT_READINESS_BINDER.md` Pilot readiness binder for client delivery

src/

src/config/

`cerbaseal-config.ts` loadCerbaSealConfig(), buildAllowedAuthorityClasses(), GateConfig interface

src/domain/types/

`core.ts` All primary types: GovernedRequest, DecisionEnvelope, ApprovalArtifact, GateResult, ReleaseAuthorization, BlockedActionRecord, DecisionProposal, ControlStatus, TrustState, PolicyPackRef, ProvenanceRef

`audit.ts` Audit types: AuditLogEntry, AuditEventPayload, EvidenceBundle, ExportManifest, ReplayResult

`diagnostics.ts` DiagnosticReport, HealthReport, SystemState, InvariantCheckSummary types

src/domain/constants/

`invariants.ts` INVARIANTS const object — INV-01 through INV-12 with descriptions

`reason-codes.ts` REASON_CODES const object — all 18 reason codes as string literal union

src/domain/errors/

`cerbaseal-error.ts` CerbaSealError class extending Error — carries invariant: InvariantCode, reasonCode: ReasonCode, finalState: 'HOLD' | 'REJECT'

src/domain/formatters/

`certificate.ts` CertificateFormatter — human-readable governance certificate from EvidenceBundle

`demo-response.ts` DemoResponseFormatter — shapes GateResult for the browser demo JSON API

`review-portal-data.ts` Data builders for the /api/review-summary, /api/pilot-readiness, /api/security-summary endpoints

src/domain/builders/

`agent-scenarios.ts` Agent gate test scenario builders

`consumer-scenarios.ts` Consumer example scenario builders

`gate-scenarios.ts` Core gate scenario builders for ALLOW, HOLD, and REJECT paths
`request-fixtures.ts` GovernedRequest fixture builders for tests — builds valid and invalid requests

src/services/execution/
`execution-gate-service.ts` ExecutionGateService class + assertIsGateIssued() + _gateIssuedResults WeakSet — the mandatory enforcement gate
`mock-execution-system.ts` MockExecutionSystem — test helper for simulating caller systems
`tools.ts` Shared gate utility functions

src/services/audit/
`append-only-log-service.ts` AppendOnlyLogService (in-memory) + IAuditLogService interface definition
`file-backed-append-only-log-service.ts` FileBackedAppendOnlyLogService — JSONL persistence using appendFileSync
`audit-hash-utils.ts` buildEntry(), verifyChain(), deepClone() — SHA-256 chain construction and verification

src/services/evidence/
`evidence-bundle-service.ts` EvidenceBundleService — assembles full decision context, calls assertIsGateIssued

src/services/export/
`export-manifest-service.ts` ExportManifestService — createAuthorityPackageManifest() — point-in-time evidence hash snapshot

src/services/replay/
`replay-service.ts` ReplayService — replay() — re-runs past decisions through a fresh gate instance

src/services/diagnostics/
`diagnostic-report-service.ts` DiagnosticReportService — createReport() — operator analysis with recommended actions

src/services/support/
`operator-action-service.ts` Operator-level actions: acknowledge, escalate, re-evaluate, generate support ticket
`system-health-service.ts` SystemHealthService — checks logging, control status, trust state, chain integrity
`system-integrity-service.ts` SystemIntegrityService — runs integrity checks across the full audit log chain
`support-fixtures.ts` Test fixtures for support service tests

test/
`execution-gate-service.test.ts` ~120 tests — all 12 invariants, all outcome paths, approval binding, timestamp, schema edge cases
`adversarial-integrity.test.ts` ~18 tests — approval replay, cross-request binding, AI escalation, malformed envelope injection
`audit-evidence-export.test.ts` ~30 tests — audit chain, evidence assembly, export manifest
`persistent-audit-log.test.ts` ~20 tests — FileBackedAppendOnlyLogService write, read-back, chain verify
`diagnostic-report-service.test.ts` ~18 tests — report structure,

recommendations, risk assessment
`security/fail-closed.test.ts` 2 tests — unexpected exceptions never produce ALLOW
`security/non-forgery.test.ts` 2 tests — self-constructed GateResult rejected by WeakSet check
`security/misuse-scenarios.test.ts` ~15 tests — AI self-auth, authority-bearing proposals, forgery attempts
`security/contextual-boundary.test.ts` ~12 tests — empty strings, null fields, whitespace-only values, ISO timestamp edge cases
`snapshots/enforcement-loop.snapshot.test.ts` ~8 tests — snapshot comparisons across full ALLOW/HOLD/REJECT flows
`integration/full-flow.test.ts` 1 test — end-to-end: request !' gate !' evidence !' replay !' chain verify
`integration/system-integration.test.ts` 1 test — full stack: gate + audit + evidence + diagnostic + health + export
`integration/browser-demo-routes.test.ts` ~40 tests — all 9 HTTP routes respond 200 with correct content
`integration/review-portal-routes.test.ts` ~40 tests — portal route content assertions
`integration/support-readiness.test.ts` ~30 tests — support readiness scenarios
`integration/external-signal-examples.test.ts` ~34 tests — trust state, control status, prohibited use, provenance permutations

examples/

`run-demo.ts` Console demo — all gate scenarios
`http-wrapper.ts` HTTP wrapper utility
`rest-api-starter/` REST API starter kit — index.ts + README.md + sample-request.json
`financial-approval-starter/` Financial approval starter kit — index.ts + README.md
`fraud-workflow-starter/` Fraud workflow starter kit — index.ts + README.md
`agent-integration-starter/` Agent integration starter kit — index.ts + README.md
`browser-demo/` HTTP review portal — server.ts, scenarios.ts, review-portal.ts, response-builder.ts, validate-demo.ts, validate-review-portal.ts
`consumer-example/` Consumer pattern — consumer.ts, mock-execution-system.ts, validate-consumer.ts, README.md
`agent-gate/` AI agent gate — agent.ts, tools.ts, run-agent-gate.ts, validate-agent-gate.ts, README.md
`auditor-view/` Auditor certificate view — render-certificate.ts, run-auditor-view.ts, validate-auditor-view.ts, README.md
`support-readiness/` Support readiness — run-support-readiness.ts, validate-support-readiness.ts

scripts/

`repo-audit.ts` 15-check automated repo audit — all checks must pass before release
`export-proof.ts` Generate docs/reports/proof-snapshot.json with SHA-256 manifest checksum + optional HMAC

`verify-proof.ts` Verify proof-snapshot.json manifest checksum
`check-imports.ts` Import boundary checker — scans 79 files for architectural violations
`check-invariant-coverage.ts` Verifies each of the 12 invariants appears in at least one test file
`generate-pilot-config.ts` Reads wizard-input.json !' writes pilot-config/ (4 files)
`generate-evidence-report.ts` Reads proof-snapshot.json !' writes evidence-package/ (3 files)
`generate-system-pdf.ts` Generates this PDF — cerbaseal-system-breakdown.pdf
`wizard-input.example.json` Fully filled example wizard-input.json for a Transaction Fraud Review workflow
`hash-report.ts` Report hash utility

docs/ (selected highlights)

`FOUNDER-INDEPENDENCE-KIT.md` Master index for all 7 adoption stages — from client qualification to compliance review
`client-adoption/onboarding-sequence.md` 8-phase client onboarding sequence (Phase 0–8) runnable without Jesse
`client-adoption/` 20+ guides: admin-guide, discovery-script, qualification-scorecard, readiness-assessment, EU AI Act mapping, pilot-delivery-playbook, pricing-framework, success-framework, quickstart-guide, troubleshooting-guide, workflow-mapping-workbook, training materials, templates (×4)
`deployment/` deployment-modes.md, eu-pilot-deployment-posture.md, mode-c-client-controlled.md, pilot-deployment-checklist.md, runbook.md
`security/` access-control-and-rate-limiting.md, artifact-signing-roadmap.md, security-review-brief.md
`reports/` adversarial integrity reports, CTO review pack (LINE_AXIA), FULL_AUDIT_REPORT_2026-06-04.md, proof-snapshot.json

artifacts/cerbaseal-demo/

`src/` React + Vite browser UI (hooks/use-toast.ts, lib/utils.ts)
`vite.config.ts` Vite config — reads PORT environment variable
`package.json` @workspace/cerbaseal-demo — registered as pnpm workspace artifact
`tsconfig.json` TypeScript config for the Vite artifact
`components.json` shadcn/ui component registry config

specs/

`approval_artifact.json` JSON schema for the ApprovalArtifact structure
`approval_artifact.md` Specification document for ApprovalArtifact including validation rules